Shark Attack Risk Analysis

Final Project Report

Steffina Jerald & Tiya Bordia
Professor Guna

1. Introduction

1.1: Problem Statement

Shark attacks always seem to show up in the news and usually create a lot of fear around going in the ocean, especially for swimmers, surfers, and divers. Even though most shark attacks aren't deadly, some of them do lead to serious or fatal injuries. What surprised us is that there isn't that much research focused on what actually makes the difference between a fatal and non-fatal attack. That's what we wanted to explore in this project. Our main question was: *what factors make a shark attack more likely to be fatal?*

Using a global dataset of shark attack records, we built a classification model to predict the likelihood of death based on features like the victim's activity, location, time of day, and shark species (when available). On top of that, we created a shark attack risk score that turns these patterns into something more practical and easy to understand for real-world situations.

1.2: Connection to the Course

This project connects to a lot of what we learned in CS 210 over the semester. We started with messy raw data and spent a lot of time cleaning and organizing it so it was actually usable. That step alone took a while because real-world data is rarely clean.

After that, we did exploratory data analysis using functions like head(), shape(), and basic summary statistics to understand what we were working with and whether the patterns made sense. Once we had a better idea of the dataset, we moved on to building prediction models, which we will go into later.

To evaluate everything, we used metrics like accuracy, precision, recall, F1 score, and confusion matrices. These helped us compare models and improve them step by step.

Beyond the technical side, we also focused on making our results understandable to other people. That's why we built an interactive dashboard, so that we can turn our analysis into something visual and easy to explore. This part was mostly just using our previous knowledge of app development and AI, but most of our effort was placed into the data science part of the project. Overall, this project basically brought together everything we learned in the course.

2. Novelty and Importance

2.1: Importance of the Project

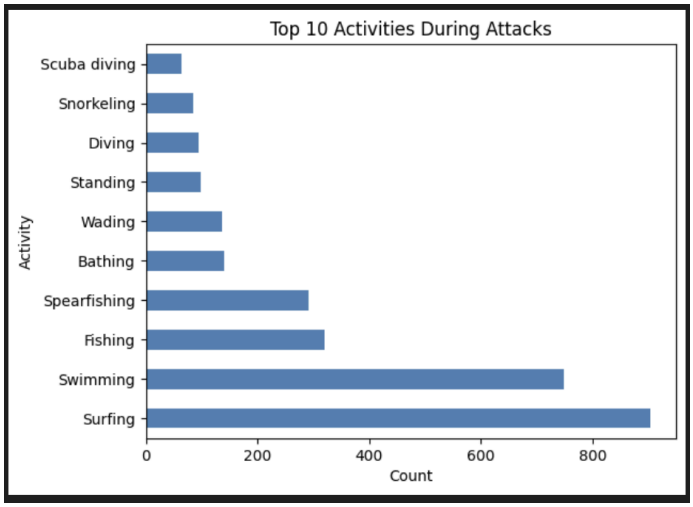
This project felt especially relevant to us because we recently joined the RU Surfing Club. Through that, we started spending more time in the ocean and learning about water safety, including shark awareness. Hearing real stories from more experienced surfers made the topic feel a lot more real. That experience is what inspired this project. We wanted to better understand shark attack risk, but also make that information more accessible so the general public could benefit from it too.

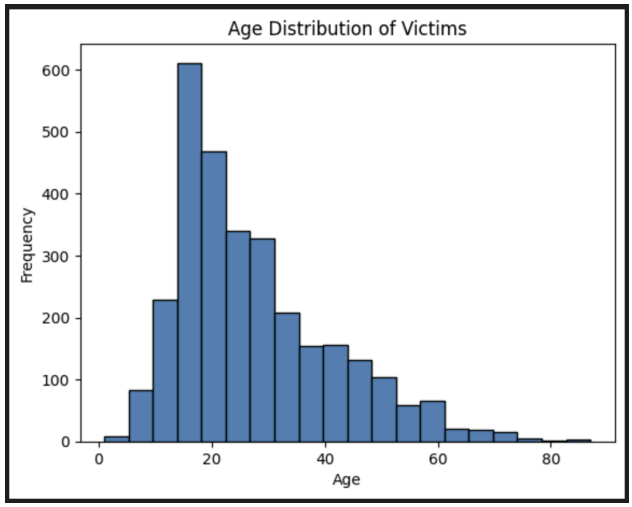
3. Methods and Results

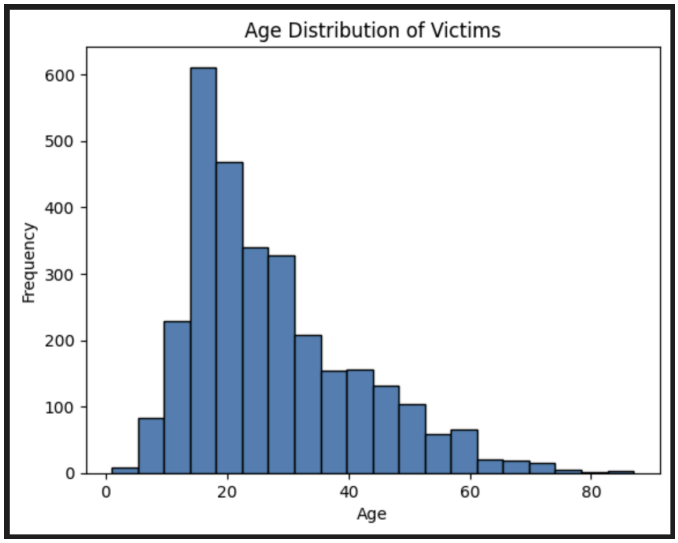
3.1: Data Utilization

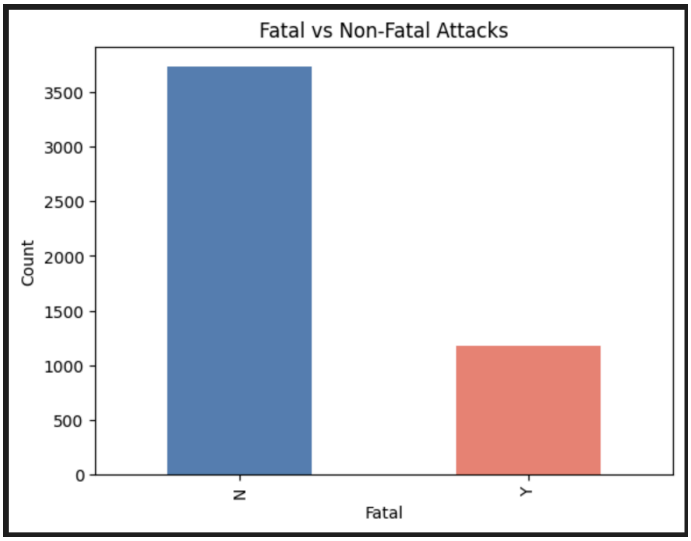
We used the Global Shark Attack Dataset, which is a public dataset containing recorded shark attack incidents from different regions and time periods. Each entry includes information like location, activity, age, sex, time of day, shark species, injury description, and whether the attack was fatal or not. Our target variable was the "Fatal (Y/N)" column.

A big part of this project was cleaning the dataset. There were a lot of missing values and inconsistent labels. For example, the same activity might be written in multiple ways, or shark species names might be abbreviated differently. We spent a lot of time standardizing these so the model could actually use the data properly.









3.2: Model and Algorithms Used

After cleaning the data, we tested three different models: Logistic Regression, Decision Tree, and Random Forest. We started with Logistic Regression because it's simple, works well for binary classification, and gives a good baseline. Then we used Decision Trees and Random Forest to see if more complex models could better handle the data, especially since predicting fatal cases turned out to be the hardest part. All models used the same features: activity, country, sex, and age.

One big issue we ran into was class imbalance, in which there were way more non-fatal attacks than fatal ones. Because of that, accuracy alone wasn't enough, so we also focused on precision, recall, and F1 score to get a better picture of performance.

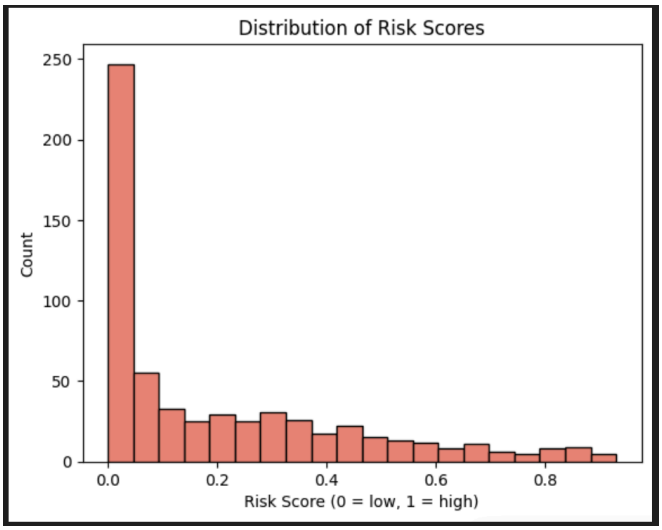
3.3: Experimental Design

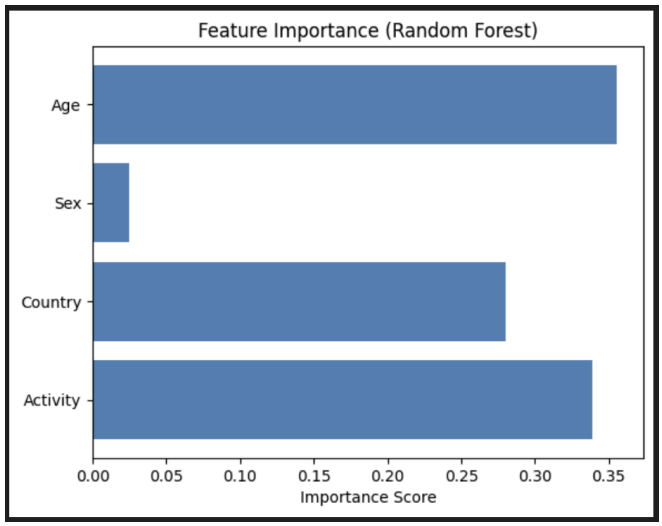
We split the dataset into 80% training data and 20% testing data. This helped us evaluate how well the models would perform on unseen data instead of just memorizing patterns. We used label encoding to convert categorical variables like activity, country, and sex into numbers so the models could process them.

After training all three models on the same dataset, we compared their performance using multiple evaluation metrics. We also used the Random Forest model later to help us understand feature importance and to generate the risk score, since it provides probability outputs instead of just a yes/no prediction.

3.4: Key Findings and Results

Some of the strongest predictors turned out to be the victim's activity, shark species (when available), and injury severity. For example, activities like diving and freediving were much more likely to be associated with fatal outcomes compared to surfing or swimming. We actually started noticing this during exploratory analysis, and the model confirmed it. We then built a risk score using the model's predicted probabilities to represent how dangerous a situation might be. Higher risk scores usually showed up in cases involving larger shark species or deeper, more isolated waters. We shared all of this through visualizations, feature importance plots, and an interactive dashboard where users can explore how different factors affect risk. Overall, some of the results were pretty surprising and showed that the situation is more complex than just general fear or assumptions about sharks.





3.6: Advantages and Limitations

One big advantage of our approach is that it's interpretable. We can actually see how different features affect predictions, which makes it much easier to explain the results in a way that makes sense. This was especially helpful when building the risk score, since our goal wasn't just to make a model that works well, but also one that people without a data science background could actually understand and use. The dashboard also made the results more accessible by turning the analysis into something visual and interactive instead of just numbers in a notebook. It lets users explore patterns on their own, which helps make the findings feel more real and intuitive.

However, there are also clear limitations to our approach. The dataset is incomplete in many areas, especially when it comes to shark species, older records, and detailed descriptions of incidents. There's also likely some reporting bias, since more severe or unusual attacks are more likely to be recorded, while minor incidents may go unreported. On top of that, we were limited by the features available in the dataset itself. Important environmental and situational factors like water visibility, ocean depth, weather conditions, wave activity, and seasonal patterns weren't included, even though they probably play a big role in real-world risk. Because of this, the model can only capture part of the full story, not the entire context behind each incident.

Another limitation is that even though models like Random Forest improve performance, they still rely heavily on the quality of the input data. So if the data is noisy or missing key variables, the predictions will always have some level of uncertainty. Also, while our risk score helps simplify the output, it is still an estimate based on historical data and shouldn't be treated as a definitive measure of danger in real-time situations.

In the future, using a more complete and consistent dataset would likely improve results a lot. Adding environmental variables and real-time ocean data could make predictions more realistic. We also think more advanced models like Gradient Boosting or ensemble methods could help capture more complex patterns and improve overall accuracy while still keeping interpretability reasonably strong.

4. Challenges

Inconsistent and messy raw data: The biggest challenge came from just how inconsistent the dataset was, especially in the "species" and "activity" columns. We kept running into different spellings, abbreviations, and incomplete labels that didn't match at all. We had to go back multiple times and re-clean the same columns as new edge cases kept showing up. It honestly felt like we couldn't even fully start modeling until we fixed things we thought were already done.

Figuring out how to group activities in a meaningful way: There wasn't a clear way to categorize activities, so a lot of it came down to judgment calls. We tried different groupings, ran the model, saw weak results, and then went back to revise them again. It was very trial-and-error until we landed on categories that actually made sense both logically and statistically.

More preprocessing than expected slowed everything down: We underestimated how much time we would spend just cleaning and standardizing the data. A lot of our early work plans had to shift because we were still stuck in preprocessing. We ended up splitting tasks and working in parallel just to keep the project moving.

Geographic data was hard to work with: Location data was all over the place and often incomplete, which made visualizing it on maps really difficult. We tried multiple approaches before deciding to simplify and filter out unreliable entries. A few of our early visualization attempts completely broke or looked misleading, so we had to redo them.

Dashboard integration was a lot of trial and error: Getting everything in the interactive dashboard to actually work together was way harder than expected. Filters would break outputs, charts wouldn't update correctly, or the risk score wouldn't sync properly. We spent a lot of time debugging, rebuilding components, and re-testing until everything finally worked smoothly.

Turning model output into a meaningful risk score: One of the hardest parts was figuring out how to translate logistic regression outputs into a single risk score that actually made sense. We tested different weighting ideas, compared outputs, and adjusted based on whether the results were interpretable or not. It took several failed versions before we got something we felt was both accurate and understandable.

Constant cycle of failing, fixing, and refining: Almost nothing worked perfectly the first time, whether it was cleaning rules, model features, or visualizations. A lot of the project involved going back, realizing something didn't work, and redoing it better. In the end, that constant iteration is really what shaped the final quality of the project.

The biggest takeaway from this part of the project is that nothing worked on the first try, but each challenge pushed us closer to a better solution.

5. Code Breakdown

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix
```

Python

This cell imports all the libraries needed to run the project. We used pandas and numpy for data handling, matplotlib and seaborn for visualizations, and scikit-learn for everything related to modeling, including the LabelEncoder, train_test_split, our three classifiers, and the evaluation metrics we used to compare them.

```python
df = pd.read_csv('attacks.csv', encoding='latin-1')

print(df.shape)
print(df.head())
print(df.columns.tolist())
print(df.dtypes)
```

This cell loads the raw shark attack dataset into a pandas DataFrame using read_csv. Before doing any cleaning or modeling, we first needed to understand what we were actually working with, so we printed out several basic properties of the dataset. The shape() told us the total number of rows and columns, while head() let us preview the first few records to get a sense of what the data looked like at a glance. We also printed the full list of column names and their data types, which helped us identify which fields were stored as strings versus numbers and where we might need to do type conversions later. This initial inspection step was important because it gave us a clear picture of the dataset's structure before we committed to any cleaning or preprocessing decisions.

```python
# Keep only useful columns
cols = ['Year', 'Country', 'Activity', 'Sex ', 'Age', 'Time', 'Species ', 'Fatal (Y/N)']
df = df[cols]

# Rename columns to remove spaces
df.columns = ['Year', 'Country', 'Activity', 'Sex', 'Age', 'Time', 'Species', 'Fatal']

# Clean the target column — keep only Y or N
df = df[df['Fatal'].isin(['Y', 'N'])]

# Convert Age to numeric (some entries are text)
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')

# Clean Sex column
df['Sex'] = df['Sex'].str.strip().str.upper()
df = df[df['Sex'].isin(['M', 'F'])]

# Drop rows missing too many key fields
df = df.dropna(subset=['Activity', 'Country'])

print(df.shape)
print(df['Fatal'].value_counts())
```

This cell handles the core data cleaning steps that were necessary before any modeling could take place. We started by selecting only the columns that were actually relevant to our analysis and renaming them to remove trailing spaces that were causing issues when referencing column names in code. The target column, Fatal, was filtered to keep only rows where the value was explicitly "Y" or "N," since there were many ambiguous or blank entries that would have introduced noise into the model. Age was converted from a string to a numeric type using pd.ti_numeric with errors='coerce', which automatically turned any non-numeric entries like "teen" or "young" into NaN rather than throwing an error. The Sex column was standardized by stripping whitespace and converting everything to uppercase, then filtered to only keep "M" or "F" entries. Finally, we dropped any rows that were missing Activity or Country values since those were two of our key features, and printed the final shape and class distribution to confirm how much data we had left after cleaning.

```python
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Fatal vs Non-Fatal counts
df['Fatal'].value_counts().plot(kind='bar', color=['steelblue', 'salmon'])
plt.title('Fatal vs Non-Fatal Attacks')
plt.xlabel('Fatal')
plt.ylabel('Count')
plt.show()

# 2. Top 10 activities
top_activities = df['Activity'].value_counts().head(10)
top_activities.plot(kind='barh', color='steelblue')
plt.title('Top 10 Activities During Attacks')
plt.xlabel('Count')
plt.show()

# 3. Fatal rate by activity
activity_fatal = df.groupby('Activity')['Fatal'].apply(lambda x: (x == 'Y').sum() / len(x))
activity_fatal = activity_fatal.sort_values(ascending=False).head(10)
activity_fatal.plot(kind='barh', color='salmon')
plt.title('Fatal Rate by Activity (Top 10)')
plt.xlabel('Fatal Rate')
plt.show()

# 4. Top 10 countries
df['Country'].value_counts().head(10).plot(kind='bar', color='steelblue')
plt.title('Top 10 Countries by Attack Count')
plt.xlabel('Country')
plt.show()

# 5. Age distribution
df['Age'].dropna().plot(kind='hist', bins=20, color='steelblue', edgecolor='black')
plt.title('Age Distribution of Victims')
plt.xlabel('Age')
plt.show()
```

Python

This cell covers our exploratory data analysis, where we created five visualizations to better understand the patterns in the dataset before building any models. The first chart was a simple bar chart comparing the count of fatal versus non-fatal attacks, which gave us an immediate look at the class imbalance we would need to account for during modeling. The second chart showed the top 10 activities during attacks by raw count, while the third went a step further by calculating the actual fatal rate for each activity. This was more informative than just counts because it revealed that activities like free diving were far more deadly proportionally even though they appeared less frequently in the data. The fourth chart displayed the top 10 countries by number of recorded attacks, and the fifth was a histogram showing the age distribution of victims across the dataset. Together, these five visuals helped us identify which variables seemed most related to fatal outcomes and guided our decisions about which features to prioritize in the model.

```python
from sklearn.preprocessing import LabelEncoder

# Select features
features = ['Activity', 'Country', 'Sex', 'Age']
target = 'Fatal'

model_df = df[features + [target]].dropna()

# Encode categorical columns
le = LabelEncoder()
model_df['Activity'] = le.fit_transform(model_df['Activity'])
model_df['Country'] = le.fit_transform(model_df['Country'])
model_df['Sex'] = le.fit_transform(model_df['Sex'])
model_df['Fatal'] = (model_df['Fatal'] == 'Y').astype(int)

X = model_df[features]
y = model_df[target]

print(X.shape)
print(y.value_counts())
```

Python

This cell prepares the data for modeling by selecting the four features we decided to use: Activity, Country, Sex, and Age and dropping any remaining rows with missing values, and using LabelEncoder to convert the categorical columns into numeric values that the models could work with. The target column Fatal was also converted into binary format, where 1 represents a fatal attack and 0 represents non-fatal, and then the data

was split into X for the input features and y for the target labels. We printed the final shape and class distribution as a last check before moving into model training.

```python
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(random_state=42)
}

for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    print(f"\n--- {name} ---")
    print(f"Accuracy: {accuracy_score(y_test, preds):.3f}")
    print(f"F1 Score: {f1_score(y_test, preds):.3f}")
    print(classification_report(y_test, preds))
```

Python

This cell handles the training and evaluation of all three models. We first split the data into an 80/20 train-test split, then trained Logistic Regression, Decision Tree, and Random Forest on the same training set so the comparison would be fair. For each model, we printed the accuracy, F1 score, and a full classification report showing precision and recall per class, which gave us a much more complete picture of performance than accuracy alone, especially important given our class imbalance where non-fatal attacks greatly outnumbered fatal ones.

```python
import numpy as np

rf_model = models['Random Forest']
importances = rf_model.feature_importances_

plt.barh(features, importances, color='steelblue')
plt.title('Feature Importance (Random Forest)')
plt.xlabel('Importance Score')
plt.show()
```

Python

This cell extracts and visualizes the feature importance scores from the Random Forest model. Random Forest calculates these scores based on how much each feature reduces impurity across all the decision trees in the

ensemble, giving us a data-driven way to see which variables mattered most in predicting fatality. The horizontal bar chart made it easy to compare all four features at a glance, and the results showed that Age and Activity were the strongest predictors, while Sex contributed almost nothing to the model's decisions.

```python
# Use predicted probabilities from Random Forest as the risk score
rf_probs = rf_model.predict_proba(X_test)[:, 1]  # probability of fatal

# Add back to a results dataframe
results = X_test.copy()
results['Risk Score'] = rf_probs
results['Actual Fatal'] = y_test.values

# Show high-risk cases
print(results.sort_values('Risk Score', ascending=False).head(10))

# Visualize score distribution
plt.hist(rf_probs, bins=20, color='salmon', edgecolor='black')
plt.title('Distribution of Risk Scores')
plt.xlabel('Risk Score (0 = low, 1 = high)')
plt.ylabel('Count')
plt.show()
```

Python

This cell generates the risk score by extracting the predicted probabilities from the Random Forest model, which returns a confidence value between 0 and 1 for each test case rather than just a hard yes or no prediction. We took the probability of the fatal class specifically and added it back to a copy of the test set alongside the actual fatal label so we could compare predicted risk against real outcomes. We then sorted the results by risk score in descending order and printed the top 10 highest-risk cases, which let us see exactly which combinations of features the model considered most dangerous. Finally, we plotted the full distribution of risk scores as a histogram, which showed that most test cases clustered toward the lower end of the scale with a smaller tail of high-risk scenarios on the right. This continuous risk score was more useful than a simple fatal or non-fatal prediction because it allowed us to rank situations by degree of danger rather than just putting them into one of two buckets.

5. Conclusion

Overall, this project was challenging but also really rewarding. It gave us a chance to apply everything we learned in a real-world context and better understand how data science can be used for risk analysis and decision-making. We also learned how to build a structured, interactive dashboard, and gained experience working through messy, real datasets from start to finish. In the future, we'd like to improve the model further, handle class imbalance more effectively, and expand the dataset with more environmental features. We'd also like to add more interactive features to the dashboard for scenario-based predictions.

Finally, we want to thank Professor Guna and our TA for their guidance and support throughout the project. Their feedback really helped us build the technical skills through this semester.